AI Gardening Assistant — Full Development Plan

A detailed roadmap for building a **serverless, self-researching AI gardening advisor** embedded inside a **Flutter application**, using **only free and local tools**.

---

1. Project Overview

Goal

Create an **AI agent** that:

- Runs entirely **on-device** (no paid APIs or servers)
- Answers **gardening questions and plant care queries**
- Can **autonomously search the web** for new info
- Includes a **baseline offline knowledge base**
- Responds **quickly** and **naturally**

---

2. System Architecture

```
Flutter Application
│
├── AI Agent Layer
│      ├ Local LLM (Phi-3-mini, Gemma-2B, etc.)
│      ├ Search Engine Tool (DuckDuckGo / Web Scraper)
│      ├ RAG Module (local gardening knowledge)
│      ├ Reasoning Engine (decides when to search)
│      └ Summarizer (concise advice generator)
│
└── Chat UI
       ├ User Input Field
       ├ Message Display (chat bubbles)
       └ Optional: Voice or Image Input
```

All intelligence runs locally on the device — the only external call is to a free web search endpoint.

---

3. Tech Stack

Component	Tool	Purpose
Framework	Flutter	Cross-platform app UI
LLM Runtime	MLC LLM or Ollama	Run open models locally
Language Model	Phi-3-mini (2.8B) / Gemma-2B	Fast, small, strong reasoning

Component	Tool	Purpose
Search Engine	DuckDuckGo API / HTML scraper	Free, anonymous web search
Retrieval Storage	JSON / SQLite	Local baseline knowledge
Parser	`html` Dart package	Extracts web page text
UI Extras	`speech_to_text`, `flutter_tts`	Voice I/O
Offline Embeddings	MiniLM / E5-small	RAG retrieval

🌱 4. Development Pipeline

Stage 1 — Base Flutter Setup

```
flutter create gardening_ai_assistant
```

Add to `pubspec.yaml`:

```
dependencies:
  http: ^1.2.0
  html: ^0.15.4
  mlc_llm:
  path_provider:
  speech_to_text:
  flutter_tts:
```

Stage 2 — Integrate Local Model (LLM)

Option A: On-device (Recommended)

- Use **MLC LLM** for Android/iOS.
- Convert a quantized model (Phi-3-mini Q4 GGUF) using MLC compiler.
- Add model to `assets/models/`.

Example:

```
import 'package:mlc_llm/mlc_llm.dart';

final model = await MLCModel.create(modelPath: 'models/phi3-mini-q4');
final reply = await model.generate("You are a gardening expert. How to grow
basil?");
```

Option B: Desktop Testing

- Use **Ollama** on desktop (`ollama run phi3-mini`).
- Communicate via `http://localhost:11434/api/generate`.

Stage 3 — Web Search Module

Using DuckDuckGo (Free API):

```
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<List<String>> searchWeb(String query) async {
  final res = await http.get(Uri.parse('https://api.duckduckgo.com/?
q=$query&format=json'));
  final data = jsonDecode(res.body);
  final List<String> results = [];
  if (data['RelatedTopics'] != null) {
    for (var t in data['RelatedTopics']) {
      if (t['Text'] != null) results.add(t['Text']);
    }
  }
  return results;
}
```

Optional: HTML Scraping

For deeper web content:

```
import 'package:html/parser.dart' as html;

Future<String> scrapePage(String url) async {
  final res = await http.get(Uri.parse(url));
  final document = html.parse(res.body);
  return document.body?.text ?? '';
}
```

Stage 4 — Reasoning Loop (Self-Researching Agent)

```
Future<String> askGardeningAgent(String userMessage) async {
  final decision = await model.generate("""
You are a gardening assistant.
If you need to look up specific or updated information, reply with [SEARCH:
topic].
```

```
Otherwise, answer directly.
User: $userMessage
""");

  if (decision.contains("[SEARCH:")) {
    final topic = RegExp(r"\[SEARCH:
(.*?)\]").firstMatch(decision)!.group(1)!.trim();
    final searchResults = await searchWeb(topic);
    final context = searchResults.join("
");

    final summary = await model.generate("""
You are a gardening expert.
Summarize the following search results into concise, helpful advice:
Context: $context
User: $userMessage
""");

    return summary;
  } else {
    return decision;
  }
}
```

Stage 5 — Add Offline Knowledge Base (Optional)

`assets/gardening_basics.json`:

```
[
  {"topic": "basil", "text": "Basil needs at least 6 hours of sun and warm
soil."},
  {"topic": "succulent", "text": "Succulents store water in their leaves; avoid
overwatering."}
]
```

Stage 6 — Flutter Chat UI

- `ListView` for messages
- `TextField` for input
- `FloatingActionButton` for sending

Each user message triggers:

```
final answer = await askGardeningAgent(userMessage);
```

Optional:

- Add **voice-to-text**
- Add **text-to-speech**
- Add **image upload** for plant identification later

Stage 7 — Optimization for Speed

Optimization	Description
Quantized model	Use Q4 or Q8 GGUF model
Caching	Keep model loaded in memory
Async tasks	Run search/summarization in isolates
Prompt trimming	Keep token context < 1024
Cache results	Store frequent queries locally

Expected latency (modern Android/iOS):
~0.5–2 seconds per short answer

Stage 8 — Testing & Evaluation

Test with categories like:

1. 🌱 General care → "How often to water basil?"
2. 🐛 Pest diagnosis → "White spots on rose leaves"
3. 🌤️ Environmental tips → "Herbs for low-light balconies"
4. 🌸 Advanced topics → "pH levels for tomato growth"

🧩 10. Design Choices Summary

Requirement	Decision
Speed	Phi-3-mini Q4
No paid APIs	DuckDuckGo + local LLM
Adaptability	Reasoning loop + RAG hybrid
Offline mode	Baseline JSON database
Cross-platform	Flutter

🌼 11. Future Improvements

- 🌤️ **Local weather integration**

- 📸 **Plant photo recognition**
- 🧠 **Persistent chat memory**
- 🌿 **Dynamic plant care scheduler**
- 💬 **Multilingual support**

🗂 Folder Structure

```
flutter_ai_gardener/
├─ lib/
│  ├─ main.dart
│  ├─ ai_agent.dart
│  ├─ web_search.dart
│  └─ rag_retriever.dart
├─ assets/
│  ├─ models/
│  │  └─ phi3-mini-q4.gguf
│  └─ gardening_basics.json
└─ pubspec.yaml
```

☑ Summary

A **complete blueprint** to build a **serverless AI gardening agent**, capable of:

- Running **fully locally**
- **Researching live** from the web
- Giving **instant, contextual advice**
- Extensible with **offline knowledge & vision**